Tarea #5

Velasco López Aarón Isaías

Debemos construir un pequeño renderizador por software que sea capaz de leer un cubo desde un archivo OBJ, usando el modelo con caras cuadradas o el modelo triangulado, y mostrarlo en una ventana. Las caras del cubo deben tener un color distinto y el objeto debe iluminarse con dos fuentes de luz, una blanca y una roja, con la posibilidad de activar una sola o ambas.
Para el renderizado del cubo se debe utilizar el modelo completo de iluminación con componente ambiental, difusa y especular, empleando sombreado Phong-Blinn, además se deben usar los parámetros de material que se asignaron y explicar en el reporte los valores ambiental y especular, junto a todo esto debemos de generar dos posiciones u orientaciones distintas de cámara y permitir cambiar entre proyección ortogonal y en perspectiva.Todo esto debe de realizarse en una región de memoria, para después aplicarle opcionalmente un kernel de filtrado y finalmente mostrar el resultado en la ventana.

Para esto usamos lo siguiente:

archivos OBJ: Usamos la lectura de archivos OBJ para cargar el cubo desde un modelo externo, estos archivos vienen los vértices y las caras del objeto, donde los vértices indican los puntos en el espacio 3D y las caras indican cómo se conectan estos puntos para formar la superficie del cubo.

Representación con vértices y caras: El cubo se representa mediante vértices con coordenadas x,y,z donde:

- x posición horizontal
- y posición vertical
- z profundidad

Los vértices forman las caras del objeto, si una cara viene como cuadrilátero, se divide en dos triángulos.

Transformaciones geométricas: Se utilizaron transformaciones geométricas para modificar la orientación del cubo dentro del espacio 3D y se aplicaron rotaciones sobre los ejes X y Y para girar el objeto y ver distintos ángulos.

Sistema de cámara: La cámara virtual se usa para representar el punto desde donde se observa la escena, la cámara tiene una posición, un punto al que está mirando y una dirección hacia arriba.

Proyección 3D a 2D: Se convierten las coordenadas 3D en coordenadas 2D para poder dibujarlas en la ventana. En el programa se usan dos proyecciones diferentes:

- Proyección ortogonal: no existe efecto de perspectiva
- Proyección en perspectiva: los objetos lejanos se ven más pequeños
-

Rasterización: Es el proceso donde los triángulos del cubo se convierten en píxeles en pantalla para rellenar las caras del objeto y no solamente dibujar sus bordes.

Buffer de color: Se utiliza en memoria para guardar la imagen antes de mostrarla en la ventana, esto nos permite tener control total sobre cada píxel del renderizado y también facilita aplicar filtros a la imagen final.

Z-buffer o buffer de profundidad: Guarda la profundidad de cada píxel para determinar qué parte del cubo está delante y cuál está detrás, evitando que se dibujen caras ocultas encima de las visibles.

Modelo de iluminación:Se utiliza para calcular cómo afecta la luz a cada parte del cubo.

Componente ambiental: Representa la luz general del ambiente, se usa para iluminar ligeramente toda la superficie, aunque no reciba luz directa.

Iluminación difusa: Representa la luz que llega a una superficie dependiendo del ángulo entre la luz y la normal de la cara (Si la superficie apunta más hacia la luz, se ilumina más, si apunta menos, se ilumina menos).

Iluminación especular: Son los brillos o reflejos intensos que aparecen cuando la luz rebota hacia la cámara, sus componentes son:

- la dirección de la luz
- la orientación de la superficie
- la posición del observador

Fuentes de iluminación: Se generaron dos fuentes de luz, una blanca y una roja para observar cómo cambia la iluminación del cubo.

Vectores normales: Se utilizan para saber hacia dónde apunta cada cara del cubo.

Producto punto: Se usa para medir qué tan alineados están dos vectores.

Sombreado Phong-Blinn: Se usa para la iluminación, se calcula directamente en cada píxel del triángulo, lo que produce una transición más suave y un resultado visual más detallado.

Back-face culling:Funciona descartando las caras traseras para evitar dibujar las caras del cubo que están volteadas en sentido opuesto a la cámara.

Filtro kernel: El filtro modifica el resultado visual recorriendo la imagen píxel por píxel y operando sobre los canales de color.

Interacción mediante teclado: Se utiliza el teclado para permitir la interacción del usuario.

WinAPI: Crea la ventana del programa, manejar los eventos del teclado, controlar el repintado de la escena y mostrar la imagen final generada en memoria.

Estructura del código

Primero incluimos las librerías, usamos windows.h para crear la ventana, procesar eventos del sistema, manejar el temporizador y dibujar la imagen final en pantalla, también vector, string, fstream y sstream para almacenar datos y leer el archivo OBJ, mientras que cmath y algorithm se utilizan para las operaciones matemáticas necesarias en las rotaciones, proyecciones, iluminación y control de valores mínimos y máximos.
Ahora definimos las constantes del tamaño de la ventana que determinan el tamaño de los buffers donde se genera la imagen, declaramos varias estructuras de datos que sirven para organizar la información del programa. Entre ellas están los vectores tridimensionales para trabajar con posiciones y direcciones en el espacio, la estructura de color para los componentes RGB, la estructura de cara para guardar los índices de los vértices que forman cada triángulo, la estructura de cámara dónde se observa la escena, la estructura de luz para las fuentes de iluminación y la estructura de vértice en pantalla.

Luego se crean las variables globales principales del programa donde se almacenan los vértices y las caras cargadas desde el archivo OBJ, además de los buffers de color, postproceso y profundidad, ahora definimos las variables de control que permiten modificar el comportamiento de la escena, en esta misma parte se definen las dos cámaras, las dos luces, los colores base de las caras del cubo y las propiedades del material, como las componentes ambiental, difusa y especular, así como el brillo que se usa en la iluminación.
Después implementamos también las funciones matemáticas básicas necesarias para todo el pipeline de renderizado al igual que las operaciones como producto punto, producto cruz, cálculo de longitud, normalización de vectores, limitación de valores entre 0 y 1 y conversión de colores en formato flotante al formato entero que utiliza el buffer de pantalla, las funciones de rotación sobre los ejes X y Y, que permiten cambiar la orientación del cubo antes de dibujarlo.

Ahora creamos la función encargada de cargar el archivo OBJ. Esta parte abre el archivo, lee línea por línea y separa la información de vértices y caras. Cuando encuentra una línea de vértice, guarda sus coordenadas en la lista correspondiente, cuando encuentra una cara, extrae los índices de los vértices y, si la cara es cuadrada, la divide en dos triángulos para poder trabajar con ella más fácilmente en la etapa de rasterización y le asignamos un índice de color para distinguir visualmente cada cara del cubo, se define la parte del manejo de buffers y transformaciones de vista se limpia el buffer de color y el z-buffer para preparar un nuevo cuadro de la animación y luego se implementa la transformación de vista, que convierte los puntos al sistema de coordenadas de la cámara usando la posición de la cámara, la dirección hacia la que apunta y el vector superior, la proyección convierte los puntos tridimensionales en coordenadas bidimensionales de pantalla y se crean dos tipos de proyección (ortogonal y en perspectiva).

La siguiente parte del código corresponde al modelo de iluminación donde se calcula el color final de cada píxel combinando la componente ambiental, la difusa y la especular usando la normal de la superficie, la dirección de cada luz, la dirección hacia la cámara y los coeficientes del material, Aplicamos el modelo Phong-Blinn, ya que la iluminación se calcula en cada píxel y no solo por cara o por vértice.

Después generamos la rasterización del triángulo donde se calcula el rectángulo mínimo que encierra al triángulo, luego se recorre píxel por píxel esa región y se verifica si el punto actual se encuentra dentro del triángulo, si el punto pertenece al triángulo, se interpolan sus valores y se calcula su profundidad y lo comparamos con el z-buffer para saber si realmente debe dibujarse o si hay otra cara más cercana a la cámara.
Si el píxel es visible, se calcula su color con la función de iluminación y se guarda en el buffer de color. Más adelante implementamos el filtro kernel, donde se copia la imagen renderizada al buffer de postproceso y luego se recorre la imagen tomando la vecindad de cada píxel para modificar sus valores de color, todo esto permite alterar el resultado visual antes de mostrarlo en la ventana.

Ahora creamos la función principal de renderizado, que es donde se unen todas las etapas, se limpian los buffers, luego se toma la cámara activa y se preparan arreglos temporales para los vértices transformados y proyectados, después se aplican las rotaciones al cubo, se transforman los vértices al espacio de cámara, se proyectan a la pantalla y se almacenan. Ahora se recorren todas las caras del objeto, se calcula su normal, se aplica el descarte de caras traseras y finalmente se rasterizan los triángulos visibles. Si el filtro está activado, al final se aplica sobre la imágen completa, se crea la función que dibuja el texto informativo en la ventana y luego se implementa el procedimiento de ventana WndProc, que es donde se procesan todos los mensajes del sistema.

En esta parte se controla la interacción con el teclado para encender o apagar luces, cambiar cámara, cambiar proyección, activar el filtro, cambiar de archivo OBJ, reiniciar la escena, controlar la rotación y cerrar la ventana, se procesa el temporizador que permite actualizar continuamente la rotación automática y el mensaje de repintado, donde se llama a la función principal de renderizado y se dibuja el contenido del buffer en la ventana con StretchDIBits. Por último la función WinMain, que es el punto de entrada del programa en una aplicación de Windows que intenta cargar el archivo OBJ inicial, luego se registra la clase de la ventana, se crea la ventana principal, se muestra en pantalla, se activa el temporizador y se inicia el ciclo principal de mensajes. El ciclo mantiene al programa en ejecución y permite que la ventana responda tanto a las acciones del usuario como a las actualizaciones necesarias para redibujar la escena.

Código Ejecutable

```
#include <windows.h>
#include <vector>
#include <string>
#include <fstream>
#include <sstream>
#include <cmath>
#include <algorithm>

// Tamaño de la ventana y del buffer de dibujo
const int WIDTH = 900;
```

```
const int HEIGHT = 700;

// Creamos la estructura para manejar los vectores en 3D
struct Vec3 {
    float x, y, z;
    Vec3() : x(0), y(0), z(0) {} // Constructor vacío
    Vec3(float X, float Y, float Z) : x(X), y(Y), z(Z) {} //
Constructor con valores
    Vec3 operator+(const Vec3& o) const { return Vec3(x + o.x, y + o.y,
z + o.z); } // Suma de vectores
    Vec3 operator-(const Vec3& o) const { return Vec3(x - o.x, y - o.y,
z - o.z); } // Resta de vectores
    Vec3 operator*(float s) const { return Vec3(x * s, y * s, z * s); }
// Multiplicación por escalar
};

// Creamos la estructura para manejar colores RGB
struct Color3 {
    float r, g, b;
    Color3() : r(0), g(0), b(0) {} // Constructor vacío
    Color3(float R, float G, float B) : r(R), g(G), b(B) {} //
Constructor con valores
    Color3 operator+(const Color3& o) const { return Color3(r + o.r, g
+ o.g, b + o.b); } // Suma de colores
};

// Generamos el producto punto entre dos vectores
static inline float Dot(const Vec3& a, const Vec3& b) { return a.x*b.x
+ a.y*b.y + a.z*b.z; }

// Generamos el producto cruz entre dos vectores
static inline Vec3 Cross(const Vec3& a, const Vec3& b) {
    return Vec3(a.y*b.z - a.z*b.y, a.z*b.x - a.x*b.z, a.x*b.y -
a.y*b.x);
}

// Genramos la longitud de un vector
static inline float Length(const Vec3& v) { return std::sqrt(Dot(v,
v)); }

// Convertimos un vector a longitud 1
static inline Vec3 Normalize(const Vec3& v) {
    float len = Length(v);
```

```
    if (len < 1e-8f) return Vec3(0,0,0);
    return v * (1.0f / len);
}

// Limitamos un valor entre 0 y 1
static inline float Clamp01(float x) {
    if (x < 0.0f) return 0.0f;
    if (x > 1.0f) return 1.0f;
    return x;
}

// Convertimos el color RGB a entero
static inline unsigned int PackColor(const Color3& c) {
    unsigned char r = (unsigned char)(Clamp01(c.r) * 255.0f);
    unsigned char g = (unsigned char)(Clamp01(c.g) * 255.0f);
    unsigned char b = (unsigned char)(Clamp01(c.b) * 255.0f);
    return (r) | (g << 8) | (b << 16);
}

// Se guarda una cara triangular usando índices de vértices
struct Face {
    int a, b, c;
    int faceColorIndex;
};

// Se guarda la información de una cámara
struct Camera {
    Vec3 eye;
    Vec3 target;
    Vec3 up;
};

// Se guarda la información de una luz
struct Light {
    bool enabled;
    Vec3 dir;
    Color3 intensity;
};

// Se guarda un vértice ya proyectado a pantalla
struct VertexScreen {
    float x, y, z;
    Vec3 world;
```

```
};

// Lista de vértices del modelo cargado
static std::vector<Vec3> g_vertices;

// Lista de caras del modelo cargado
static std::vector<Face> g_faces;

// Buffer principal de color
static unsigned int g_colorBuffer[HEIGHT][WIDTH];

// Buffer para la imagen filtrada
static unsigned int g_postBuffer[HEIGHT][WIDTH];

// Buffer de profundidad para z-buffer
static float g_depthBuffer[HEIGHT][WIDTH];

// Variables globales de estado
static bool g_usePerspective = true;
static bool g_useFilter = false;
static bool g_rotate = false;
static bool g_useTrianglesOBJ = true;
static int g_cameraIndex = 0;
static float g_yaw = 0.8f;
static float g_pitch = -0.3f;
static float g_autoAngle = 0.0f;

// Creamos las dos cámaras para el cubo
static Camera g_cameras[2] = {
    { Vec3(45.0f, 30.0f, 45.0f), Vec3(0,0,0), Vec3(0,1,0) },
    { Vec3(-50.0f, 25.0f, 35.0f), Vec3(0,0,0), Vec3(0,1,0) }
};

// Generamos las luces blanca y roja
static Light g_lights[2] = {
    { true, Normalize(Vec3(-1.0f, -1.0f, -1.0f)), Color3(0.8f, 0.8f,
0.8f) },
    { true, Normalize(Vec3(-0.2f, -0.3f, -1.0f)), Color3(0.8f, 0.0f,
0.0f) }
};

// Colores de las caras del cubo
static Color3 g_faceColors[6] = {
```

```
    Color3(1.0f, 0.2f, 0.2f),
    Color3(0.2f, 1.0f, 0.2f),
    Color3(0.2f, 0.2f, 1.0f),
    Color3(1.0f, 1.0f, 0.2f),
    Color3(1.0f, 0.2f, 1.0f),
    Color3(0.2f, 1.0f, 1.0f)
};

// Coeficientes del material para iluminación
static Color3 g_ka(0.3f, 0.3f, 0.3f);
static Color3 g_kd(0.07568f, 0.61424f, 0.07568f);
static Color3 g_ks(0.633f, 0.727811f, 0.633f);
static float g_shininess = 76.8f;

// Rotación de un vector alrededor del eje X
Vec3 RotateX(const Vec3& v, float a) {
    float c = std::cos(a), s = std::sin(a);
    return Vec3(v.x, v.y*c - v.z*s, v.y*s + v.z*c);
}

// Rotación de un vector alrededor del eje Y
Vec3 RotateY(const Vec3& v, float a) {
    float c = std::cos(a), s = std::sin(a);
    return Vec3(v.x*c + v.z*s, v.y, -v.x*s + v.z*c);
}

// Cargamos el archivo OBJ y guardamos los vértices y caras
bool LoadOBJ(const char* filename) {
    std::ifstream file(filename);
    if (!file.is_open()) return false;
    g_vertices.clear();
    g_faces.clear();
    std::string line;
    int faceCounter = 0;
    while (std::getline(file, line)) {
        if (line.size() < 2) continue;
        std::stringstream ss(line);
        std::string tag;
        ss >> tag;
        if (tag == "v") {
            float x, y, z;
            ss >> x >> y >> z;
            g_vertices.push_back(Vec3(x, y, z));
```

```
        } else if (tag == "f") {
            std::vector<int> idx;
            std::string token;
            while (ss >> token) {
                size_t slash = token.find('/');
                if (slash != std::string::npos) token = token.substr(0,
slash);
                idx.push_back(std::stoi(token) - 1);
            }
            int colorIndex = faceCounter % 6;
            if (idx.size() == 3) {
                g_faces.push_back({ idx[0], idx[1], idx[2], colorIndex
});
                faceCounter++;
            } else if (idx.size() == 4) {
                g_faces.push_back({ idx[0], idx[1], idx[2], colorIndex
});
                g_faces.push_back({ idx[0], idx[2], idx[3], colorIndex
});
                faceCounter++;
            }
        }
    }
    return !g_vertices.empty() && !g_faces.empty();
}

// Limpiamos los buffers para dibujar un nuevo frame
void ClearBuffers() {
    unsigned int bg = PackColor(Color3(0.08f, 0.08f, 0.10f));
    for (int y = 0; y < HEIGHT; ++y) {
        for (int x = 0; x < WIDTH; ++x) {
            g_colorBuffer[y][x] = bg;
            g_depthBuffer[y][x] = 1e30f;
        }
    }
}

// Crea un punto del mundo al espacio de la cámara
Vec3 ViewTransform(const Vec3& p, const Camera& cam) {
    Vec3 f = Normalize(cam.target - cam.eye);
    Vec3 r = Normalize(Cross(f, cam.up));
    Vec3 u = Cross(r, f);
    Vec3 q = p - cam.eye;
```

```
    return Vec3(Dot(q, r), Dot(q, u), Dot(q, f));
}

// Proyecta un punto 3D a coordenadas 2D
bool ProjectPoint(const Vec3& camP, float& sx, float& sy, float& sz) {
    if (g_usePerspective) {
        if (camP.z <= 0.01f) return false;
        float f =700.0f;
        sx = (camP.x / camP.z) * f + WIDTH * 0.5f;
        sy = (-camP.y / camP.z) * f + HEIGHT * 0.5f;
        sz = camP.z;
        return true;
    } else {
        float scale = 12.0f;
        sx = camP.x * scale + WIDTH * 0.5f;
        sy = -camP.y * scale + HEIGHT * 0.5f;
        sz = camP.z;
        return true;
    }
}

// Calculamos el color final de un píxel con Blinn-Phong
Color3 ShadePixel(const Vec3& P, const Vec3& N, const Color3&
faceColor, const Camera& cam) {
    Vec3 normal = Normalize(N);
    Vec3 V = Normalize(cam.eye - P);
    Color3 result(faceColor.r * g_ka.r, faceColor.g * g_ka.g,
faceColor.b * g_ka.b);

    for (int i = 0; i < 2; ++i) {
        if (!g_lights[i].enabled) continue;

        Vec3 L = Normalize(g_lights[i].dir * -1.0f);
        float ndotl = std::max(0.0f, Dot(normal, L));
        Vec3 H = Normalize(L + V);
        float ndoth = std::max(0.0f, Dot(normal, H));
        float spec = std::pow(ndoth, g_shininess);
        result = result + Color3(
            faceColor.r * g_kd.r * g_lights[i].intensity.r * ndotl +
g_ks.r * g_lights[i].intensity.r * spec,
            faceColor.g * g_kd.g * g_lights[i].intensity.g * ndotl +
g_ks.g * g_lights[i].intensity.g * spec,
```

```
            faceColor.b * g_kd.b * g_lights[i].intensity.b * ndotl +
g_ks.b * g_lights[i].intensity.b * spec
        );
    }
    return Color3(Clamp01(result.r), Clamp01(result.g),
Clamp01(result.b));
}

// Creamos una función auxiliar para rasterizar triángulos con edge
function
float Edge(float ax, float ay, float bx, float by, float px, float py)
{
    return (px - ax)*(by - ay) - (py - ay)*(bx - ax);
}

// Ahora dibujamos un triángulo rellenándolo
void DrawTriangle(const VertexScreen& v0, const VertexScreen& v1, const
VertexScreen& v2,
                  const Vec3& faceNormal, const Color3& faceColor,
const Camera& cam) {
    float minX = std::max(0.0f, std::floor(std::min(v0.x,
std::min(v1.x, v2.x))));
    float maxX = std::min((float)WIDTH - 1, std::ceil(std::max(v0.x,
std::max(v1.x, v2.x))));
    float minY = std::max(0.0f, std::floor(std::min(v0.y,
std::min(v1.y, v2.y))));
    float maxY = std::min((float)HEIGHT - 1, std::ceil(std::max(v0.y,
std::max(v1.y, v2.y))));

    float area = Edge(v0.x, v0.y, v1.x, v1.y, v2.x, v2.y);
    if (std::fabs(area) < 1e-8f) return;

    for (int y = (int)minY; y <= (int)maxY; ++y) {
        for (int x = (int)minX; x <= (int)maxX; ++x) {
            float px = x + 0.5f;
            float py = y + 0.5f;
            float w0 = Edge(v1.x, v1.y, v2.x, v2.y, px, py);
            float w1 = Edge(v2.x, v2.y, v0.x, v0.y, px, py);
            float w2 = Edge(v0.x, v0.y, v1.x, v1.y, px, py);
            bool inside = (area > 0) ? (w0 >= 0 && w1 >= 0 && w2 >= 0)
: (w0 <= 0 && w1 <= 0 && w2 <= 0);
            if (!inside) continue;
            w0 /= area; w1 /= area; w2 /= area;
```

```
            float z = v0.z*w0 + v1.z*w1 + v2.z*w2;
            if (z < g_depthBuffer[y][x]) {
                g_depthBuffer[y][x] = z;
                Vec3 P = v0.world*w0 + v1.world*w1 + v2.world*w2;
                g_colorBuffer[y][x] = PackColor(ShadePixel(P,
faceNormal, faceColor, cam));
            }
        }
    }
}

// Aplicamos un filtro kernel a la imagen final
void ApplyKernel() {
    for (int y = 0; y < HEIGHT; ++y)
        for (int x = 0; x < WIDTH; ++x)
            g_postBuffer[y][x] = g_colorBuffer[y][x];
    for (int y = 1; y < HEIGHT - 1; ++y) {
        for (int x = 1; x < WIDTH - 1; ++x) {
            float sumR = 0, sumG = 0, sumB = 0;
            for (int ky = -1; ky <= 1; ++ky) {
                for (int kx = -1; kx <= 1; ++kx) {
                    unsigned int c = g_colorBuffer[y + ky][x + kx];
                    sumR += ((c) & 255) / 255.0f;
                    sumG += ((c >> 8) & 255) / 255.0f;
                    sumB += ((c >> 16) & 255) / 255.0f;
                }
            }
            unsigned int c = g_colorBuffer[y][x];
            float cr = ((c) & 255) / 255.0f;
            float cg = ((c >> 8) & 255) / 255.0f;
            float cb = ((c >> 16) & 255) / 255.0f;
            g_postBuffer[y][x] = PackColor(Color3(
                Clamp01(2.0f * cr - sumR / 9.0f),
                Clamp01(2.0f * cg - sumG / 9.0f),
                Clamp01(2.0f * cb - sumB / 9.0f)
            ));
        }
    }
}

// Renderizamos la escena
void RenderScene() {
    ClearBuffers();
```

```
    Camera cam = g_cameras[g_cameraIndex];
    std::vector<Vec3> worldVerts(g_vertices.size());
    std::vector<VertexScreen> scrVerts(g_vertices.size());
    float ay = g_yaw + (g_rotate ? g_autoAngle : 0.0f);
    float ax = g_pitch;
    for (size_t i = 0; i < g_vertices.size(); ++i) {
        Vec3 p = RotateX(RotateY(g_vertices[i], ay), ax);
        worldVerts[i] = p;
        Vec3 cp = ViewTransform(p, cam);
        float sx, sy, sz;
        if (!ProjectPoint(cp, sx, sy, sz))
            scrVerts[i] = { -99999, -99999, 1e30f, p };
        else
            scrVerts[i] = { sx, sy, sz, p };
    }
    for (const Face& f : g_faces) {
        Vec3 p0 = worldVerts[f.a];
        Vec3 p1 = worldVerts[f.b];
        Vec3 p2 = worldVerts[f.c];
        Vec3 normal = Normalize(Cross(p1 - p0, p2 - p0));
        Vec3 viewDir = Normalize(cam.eye - p0);
        if (Dot(normal, viewDir) <= 0.0f) continue;
        const VertexScreen& s0 = scrVerts[f.a];
        const VertexScreen& s1 = scrVerts[f.b];
        const VertexScreen& s2 = scrVerts[f.c];
        if (s0.z > 1e20f || s1.z > 1e20f || s2.z > 1e20f) continue;
        DrawTriangle(s0, s1, s2, normal, g_faceColors[f.faceColorIndex
% 6], cam);
    }
    if (g_useFilter) ApplyKernel();
}

// Dibujamos textos en la pantalla para que sea mas facil la
visualización de cada cosa
void DrawTextInfo(HDC hdc) {
    SetBkMode(hdc, TRANSPARENT);
    SetTextColor(hdc, RGB(230,230,230));

    std::string t1 = "1: Luz blanca  2: Luz roja  L: Solo luz roja  C:
Camara  P: Proyeccion";
    std::string t2 = "F: Filtro  O: Cambiar OBJ  ESPACIO: Rotacion
Flechas: Girar  R: Reiniciar  ESC: Salir";
```

```
    std::string t3 = std::string("OBJ: ") + (g_useTrianglesOBJ ?
"Cube_Triangles.obj" : "Cube_Quads.obj")
        + " | Camara: " + (g_cameraIndex == 0 ? "1" : "2")
        + " | Proyeccion: " + (g_usePerspective ? "Perspectiva" :
"Ortogonal");
    TextOutA(hdc, 10, 10, t1.c_str(), (int)t1.size());
    TextOutA(hdc, 10, 30, t2.c_str(), (int)t2.size());
    TextOutA(hdc, 10, 50, t3.c_str(), (int)t3.size());
}

// Es el procedimiento principal
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM
lParam) {
    switch (msg) {
    case WM_KEYDOWN:
        switch (wParam) {
        case '1': g_lights[0].enabled = !g_lights[0].enabled;
InvalidateRect(hwnd, NULL, FALSE); break; // Activa/desactiva luz
blanca
        case '2': g_lights[1].enabled = !g_lights[1].enabled;
InvalidateRect(hwnd, NULL, FALSE); break; // Activa/desactiva luz roja
        case 'C': g_cameraIndex = 1 - g_cameraIndex;
InvalidateRect(hwnd, NULL, FALSE); break;          // Cambia cámara
        case 'P': g_usePerspective = !g_usePerspective;
InvalidateRect(hwnd, NULL, FALSE); break;       // Cambia proyección
        case 'F': g_useFilter = !g_useFilter; InvalidateRect(hwnd,
NULL, FALSE); break;                  // Activa filtro
        case 'O':
            g_useTrianglesOBJ = !g_useTrianglesOBJ;
            LoadOBJ(g_useTrianglesOBJ ? "Cube_Triangles.obj" :
"Cube_Quads.obj");
            InvalidateRect(hwnd, NULL, FALSE);
            break;
        case 'L':
            g_lights[0].enabled = false;
            g_lights[1].enabled = true;
            InvalidateRect(hwnd, NULL, FALSE);
            break;
        case 'R':
            g_yaw = 0.8f;
            g_pitch = -0.3f;
            g_autoAngle = 0.0f;
            g_cameraIndex = 0;
```

```
            g_usePerspective = true;
            g_useFilter = false;
            g_rotate = false;
            g_lights[0].enabled = true;
            g_lights[1].enabled = true;
            InvalidateRect(hwnd, NULL, FALSE);
            break;
        case VK_SPACE: g_rotate = !g_rotate; InvalidateRect(hwnd, NULL,
FALSE); break;
        case VK_LEFT:  g_yaw -= 0.10f; InvalidateRect(hwnd, NULL,
FALSE); break;
        case VK_RIGHT: g_yaw += 0.10f; InvalidateRect(hwnd, NULL,
FALSE); break;
        case VK_UP:    g_pitch -= 0.10f; InvalidateRect(hwnd, NULL,
FALSE); break;
        case VK_DOWN:  g_pitch += 0.10f; InvalidateRect(hwnd, NULL,
FALSE); break;
        case VK_ESCAPE: DestroyWindow(hwnd); break;
        }
        return 0;
    case WM_TIMER:
        if (g_rotate) {
            g_autoAngle += 0.015f;
            InvalidateRect(hwnd, NULL, FALSE);
        }
        return 0;
    case WM_PAINT: {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hwnd, &ps);
        RenderScene();
        BITMAPINFO bmi = {};
        bmi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
        bmi.bmiHeader.biWidth = WIDTH;
        bmi.bmiHeader.biHeight = -HEIGHT;
        bmi.bmiHeader.biPlanes = 1;
        bmi.bmiHeader.biBitCount = 32;
        bmi.bmiHeader.biCompression = BI_RGB;

        const void* pixels = g_useFilter ? (const void*)g_postBuffer :
(const void*)g_colorBuffer;
        StretchDIBits(hdc, 0, 0, WIDTH, HEIGHT, 0, 0, WIDTH, HEIGHT,
pixels, &bmi, DIB_RGB_COLORS, SRCCOPY); // Dibuja buffer
```

```
        DrawTextInfo(hdc);
        EndPaint(hwnd, &ps);
        return 0;
    }

    case WM_DESTROY:
        KillTimer(hwnd, 1);
        PostQuitMessage(0);
        return 0;
    }
    return DefWindowProc(hwnd, msg, wParam, lParam);
}

// Función principal del programa
int WINAPI WinMain(HINSTANCE hInst, HINSTANCE, LPSTR, int nShowCmd) {
    if (!LoadOBJ("Cube_Triangles.obj")) { //
        MessageBoxA(NULL, "No se pudo abrir Cube_Triangles.obj en la
misma carpeta del ejecutable.", "Error", MB_ICONERROR);
        return 0;
    }

    WNDCLASSA wc = {};
    wc.style = CS_HREDRAW | CS_VREDRAW;
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInst;
    wc.lpszClassName = "Tarea5WinAPIV2Class";
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    RegisterClassA(&wc);
    HWND hwnd = CreateWindowA(
        wc.lpszClassName,
        "Tarea 5 - Cubo OBJ, Blinn-Phong, WinAPI",
        WS_OVERLAPPEDWINDOW,
        CW_USEDEFAULT, CW_USEDEFAULT,
        WIDTH + 16, HEIGHT + 39,
        NULL, NULL, hInst, NULL
    );
    ShowWindow(hwnd, nShowCmd);
    UpdateWindow(hwnd);
    SetTimer(hwnd, 1, 16, NULL);
    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
```

```
    }
    return 0;
}
```

Ejecutable

Para crear el ejecutable una vez tengamos todo el código hecho simplemente nos vamos a la carpeta donde esté el .c y usamos el siguiente comando en la terminal que abrimos en la carpeta:

g++ tarea5.cpp -std=c++17 -O2 -lgdi32 -o tarea5.exe

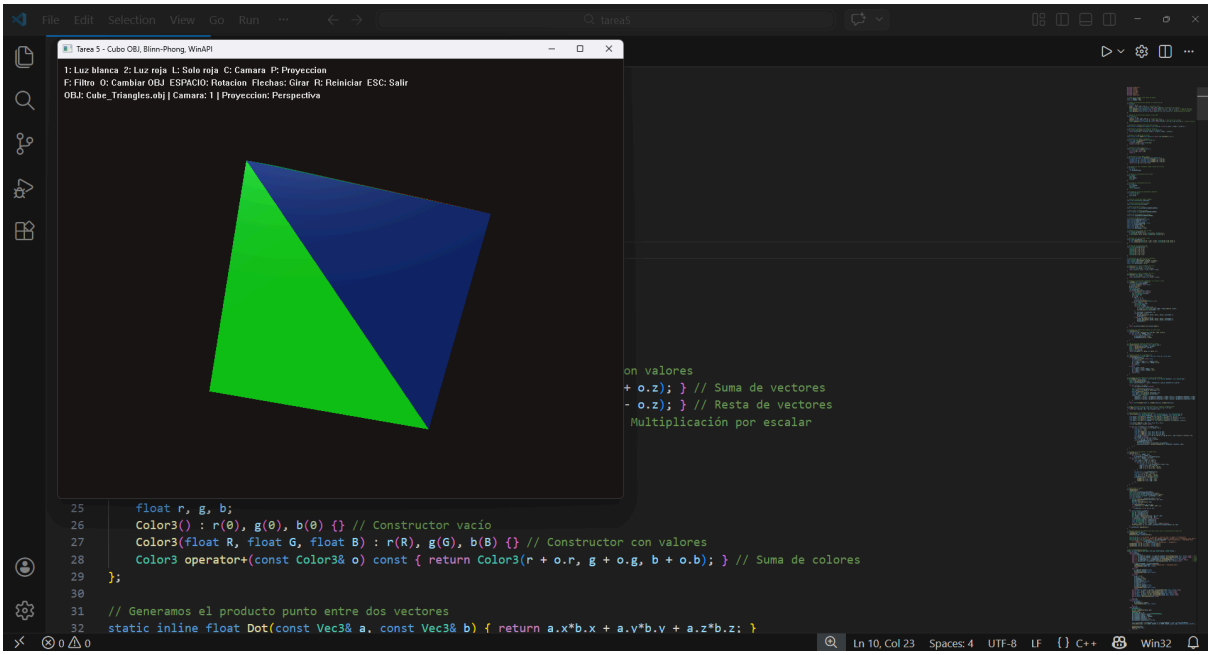
Esta línea nos creará el ejecutable (.exe) que nos permitirá ver en pantalla lo que hace el código, si queremos correr el ejecutable tendremos que poner en la terminal:

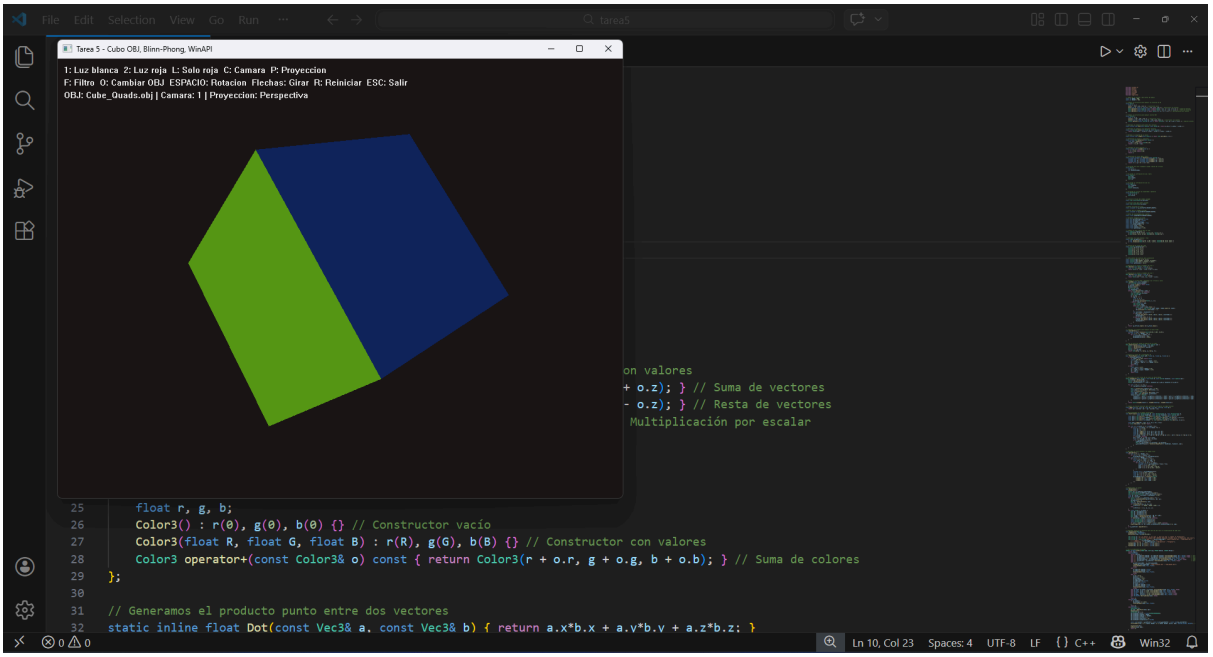
.\tarea5.exe

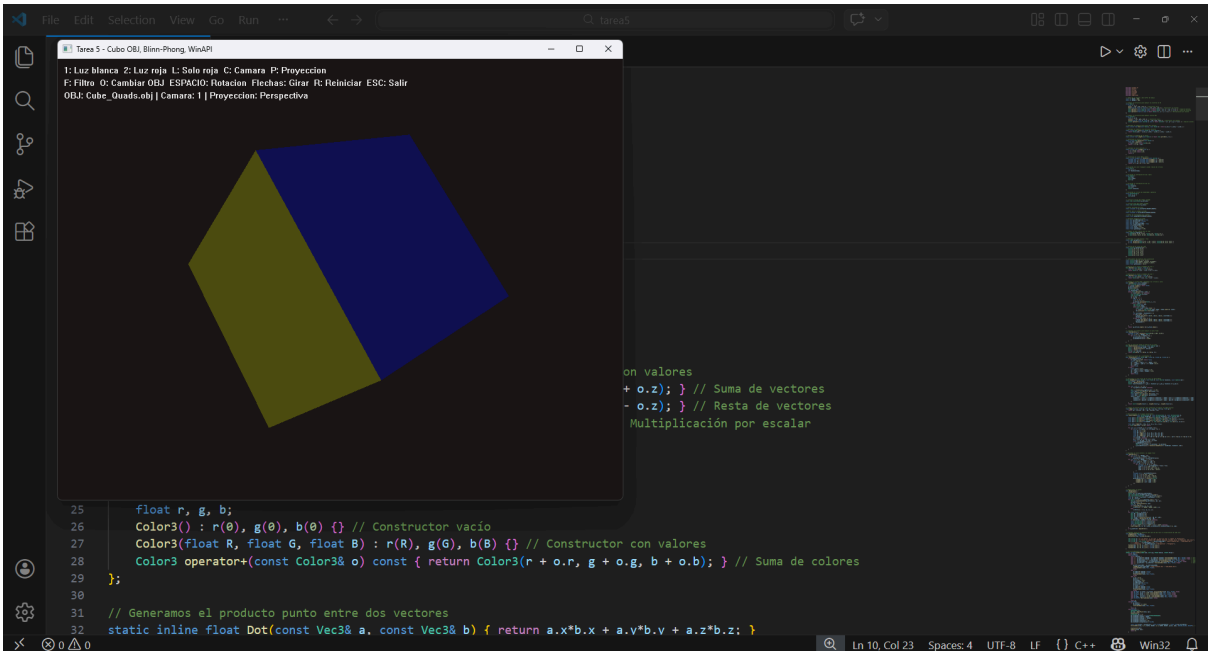
Uso del programa

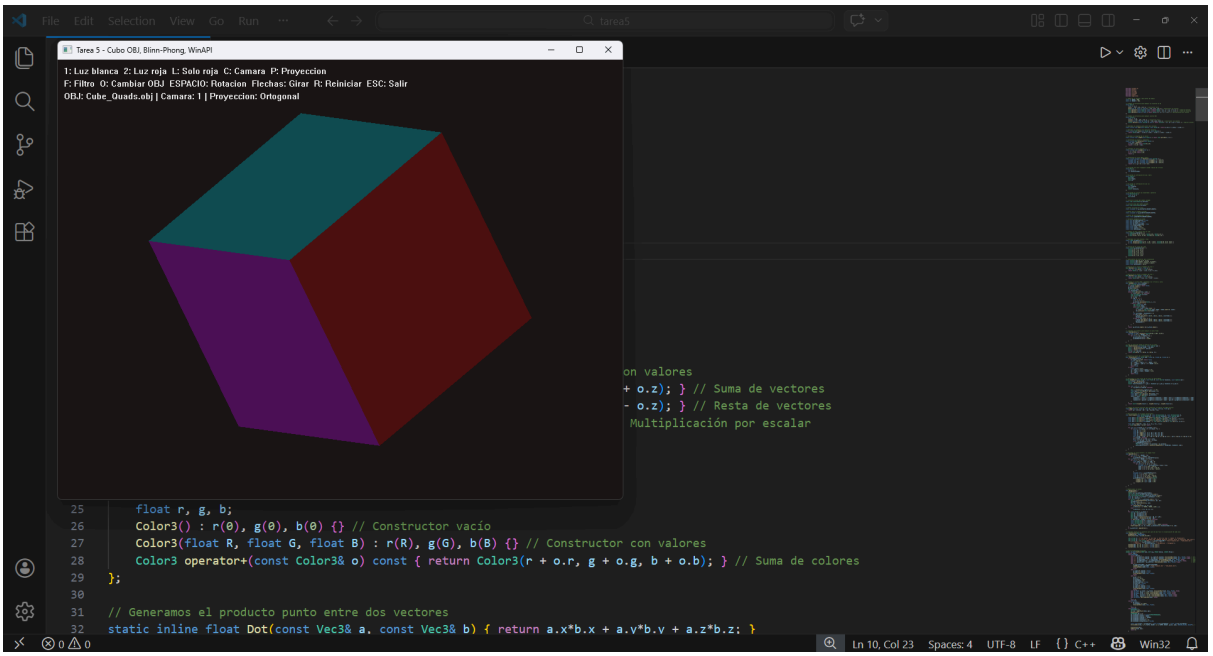
Usamos 1, 2, L, C, P, F, O, R, ESPACIO, FLECHAS, ESC para poder usar las funciones que vienen implementadas en el programa, 1 activa o desactiva la luz blanca, 2 activa o desactiva la luz roja y L deja solo la luz roja para observar mejor cómo es la luz roja contra el objeto, C cambia entre cámara 1 y cámara 2, P cambia entre proyección en perspectiva y ortogonal, F activa o desactiva el filtro kernel, O cambia entre Cube_Triangles.obj y Cube_Quads.obj, Rreinicia la escena a su estado inicial, Espacio activa o desactiva la rotación automática, Flecha izquierda gira el cubo a la izquierda, Flecha derecha gira el cubo a la derecha, Flecha arriba gira el cubo hacia arriba, Flecha abajo gira el cubo hacia abajo y Esc cierra el programa.

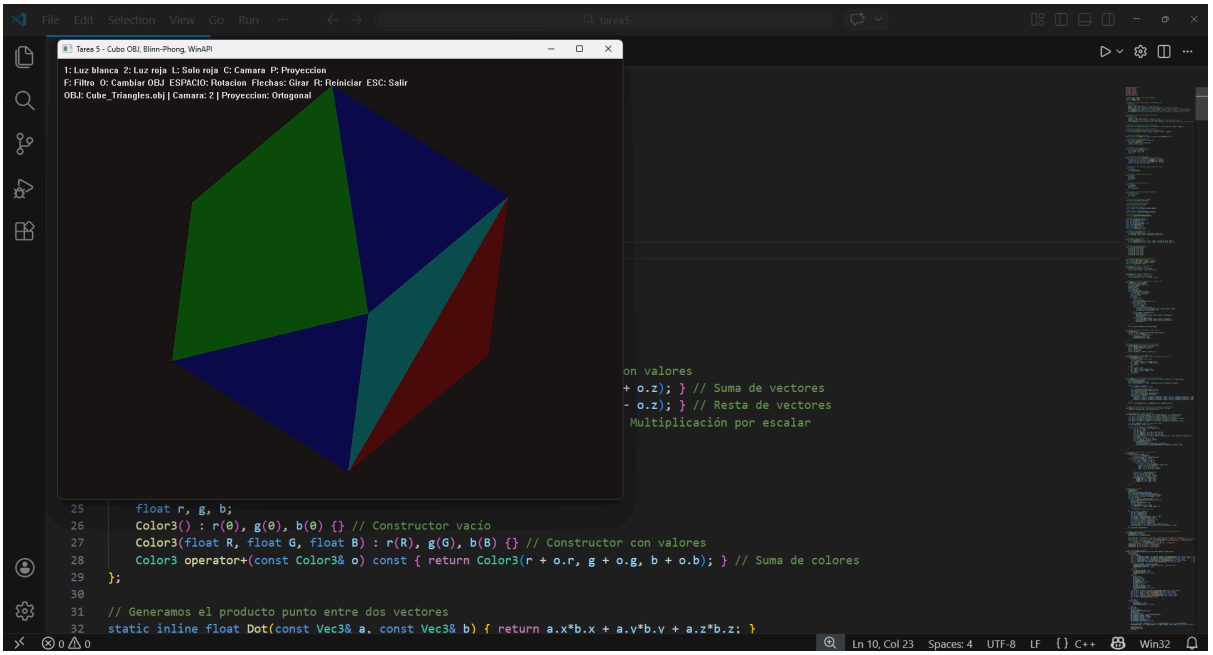
Imágenes

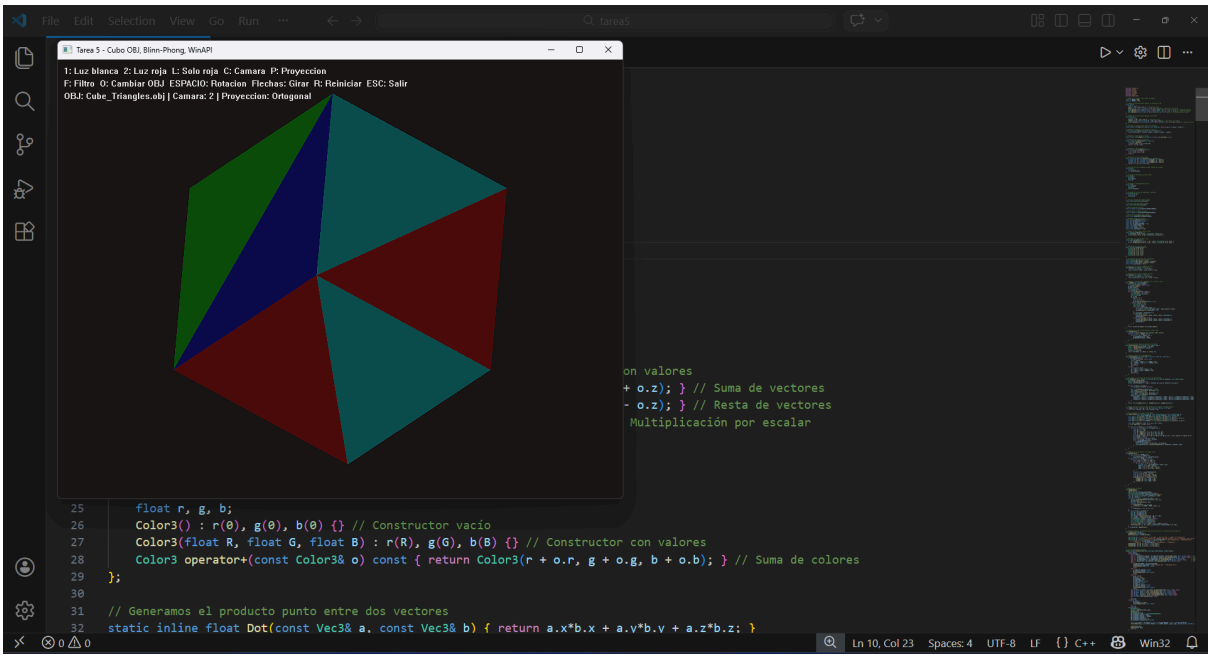

File Edit Selection View Go Run
tarea5
Tarea 5 - Cubo OBJ, Blinn-Phong, WinAPI
1: Luz blanca 2: Luz roja L: Solo roja C: Camara P: Proyeccion
F: Filtro O: Cambiar OBJ ESPACIO: Rotacion Flechas: Girar R: Reiniciar ESC: Salir
OBJ: Cube_Triangles.obj | Camara: 1 | Proyeccion: Perspectiva
on valores
+ o.z); } // Suma de vectores
- o.z); } // Resta de vectores
Multiplicación por escalar
25 float r, g, b;
26 Color3() : r(0), g(0), b(0) {} // Constructor vacio
27 Color3(float R, float G, float B) : r(R), g(G), b(B) {} // Constructor con valores
28 Color3 operator+(const Color3& o) const { return Color3(r + o.r, g + o.g, b + o.b); } // Suma de colores
29 };
30
31 // Generamos el producto punto entre dos vectores
32 static inline float Dot(const Vec3& a, const Vec3& b) { return a.x*b.x + a.y*b.y + a.z*b.z; }
Ln 10, Col 23 Spaces: 4 UTF-8 LF {} C++ Win32


File Edit Selection View Go Run
tarea5
Tarea 5 - Cubo OBJ, Blinn-Phong, WinAPI
1: Luz blanca 2: Luz roja L: Solo roja C: Camara P: Proyeccion
F: Filtro O: Cambiar OBJ ESPACIO: Rotacion Flechas: Girar R: Reiniciar ESC: Salir
OBJ: Cube_Quads.obj | Camara: 1 | Proyeccion: Perspectiva
on valores
+ o.z); } // Suma de vectores
- o.z); } // Resta de vectores
Multiplicación por escalar
25 float r, g, b;
26 Color3() : r(0), g(0), b(0) {} // Constructor vacio
27 Color3(float R, float G, float B) : r(R), g(G), b(B) {} // Constructor con valores
28 Color3 operator+(const Color3& o) const { return Color3(r + o.r, g + o.g, b + o.b); } // Suma de colores
29 };
30
31 // Generamos el producto punto entre dos vectores
32 static inline float Dot(const Vec3& a, const Vec3& b) { return a.x*b.x + a.y*b.y + a.z*b.z; }
Ln 10, Col 23 Spaces: 4 UTF-8 LF {} C++ Win32


Tarea 5 - Cubo OBJ, Blinn-Phong, WinAPI
1: Luz blanca 2: Luz roja L: Solo roja C: Camara P: Proyeccion
F: Filtro O: Cambiar OBJ ESPACIO: Rotacion Flechas: Girar R: Reiniciar ESC: Salir
OBJ: Cube_Quads.obj | Camara: 1 | Proyeccion: Perspectiva
on valores
+ o.z); } // Suma de vectores
- o.z); } // Resta de vectores
Multiplicación por escalar
25 float r, g, b;
26 Color3() : r(0), g(0), b(0) {} // Constructor vacío
27 Color3(float R, float G, float B) : r(R), g(G), b(B) {} // Constructor con valores
28 Color3 operator+(const Color3& o) const { return Color3(r + o.r, g + o.g, b + o.b); } // Suma de colores
29 };
30
31 // Generamos el producto punto entre dos vectores
32 static inline float Dot(const Vec3& a, const Vec3& b) { return a.x*b.x + a.y*b.y + a.z*b.z; }
Ln 10, Col 23 Spaces: 4 UTF-8 LF C++ Win32


Tarea 5 - Cubo OBJ, Blinn-Phong, WinAPI
1: Luz blanca 2: Luz roja L: Solo roja C: Camara P: Proyeccion
F: Filtro O: Cambiar OBJ ESPACIO: Rotacion Flechas: Girar R: Reiniciar ESC: Salir
OBJ: Cube_Quads.obj | Camara: 1 | Proyeccion: Ortogonal
on valores
+ o.z); } // Suma de vectores
- o.z); } // Resta de vectores
Multiplicación por escalar
25 float r, g, b;
26 Color3() : r(0), g(0), b(0) {} // Constructor vacío
27 Color3(float R, float G, float B) : r(R), g(G), b(B) {} // Constructor con valores
28 Color3 operator+(const Color3& o) const { return Color3(r + o.r, g + o.g, b + o.b); } // Suma de colores
29 };
30
31 // Generamos el producto punto entre dos vectores
32 static inline float Dot(const Vec3& a, const Vec3& b) { return a.x*b.x + a.y*b.y + a.z*b.z; }
Ln 10, Col 23 Spaces: 4 UTF-8 LF C++ Win32


Tarea 5 - Cubo OBJ, Blinn-Phong, WinAPI
1: Luz blanca 2: Luz roja L: Solo roja C: Camara P: Proyeccion
F: Filtro O: Cambiar OBJ ESPACIO: Rotacion Flechas: Girar R: Reiniciar ESC: Salir
OBJ: Cube_Triangles.obj | Camara: 2 | Proyeccion: Ortogonal
on valores
+ o.z); } // Suma de vectores
- o.z); } // Resta de vectores
Multiplicación por escalar
float r, g, b;
Color3() : r(0), g(0), b(0) {} // Constructor vacío
Color3(float R, float G, float B) : r(R), g(G), b(B) {} // Constructor con valores
Color3 operator+(const Color3& o) const { return Color3(r + o.r, g + o.g, b + o.b); } // Suma de colores
};
// Generamos el producto punto entre dos vectores
static inline float Dot(const Vec3& a, const Vec3& b) { return a.x*b.x + a.y*b.y + a.z*b.z; }
Ln 10, Col 23
Spaces: 4
UTF-8
LF
C++
Win32


Tarea 5 - Cubo OBJ, Blinn-Phong, WinAPI
1: Luz blanca 2: Luz roja L: Solo roja C: Camara P: Proyeccion
F: Filtro O: Cambiar OBJ ESPACIO: Rotacion Flechas: Girar R: Reiniciar ESC: Salir
OBJ: Cube_Triangles.obj | Camara: 2 | Proyeccion: Ortogonal
on valores
+ o.z); } // Suma de vectores
- o.z); } // Resta de vectores
Multiplicación por escalar
float r, g, b;
Color3() : r(0), g(0), b(0) {} // Constructor vacío
Color3(float R, float G, float B) : r(R), g(G), b(B) {} // Constructor con valores
Color3 operator+(const Color3& o) const { return Color3(r + o.r, g + o.g, b + o.b); } // Suma de colores
};
// Generamos el producto punto entre dos vectores
static inline float Dot(const Vec3& a, const Vec3& b) { return a.x*b.x + a.y*b.y + a.z*b.z; }
Ln 10, Col 23
Spaces: 4
UTF-8
LF
C++
Win32